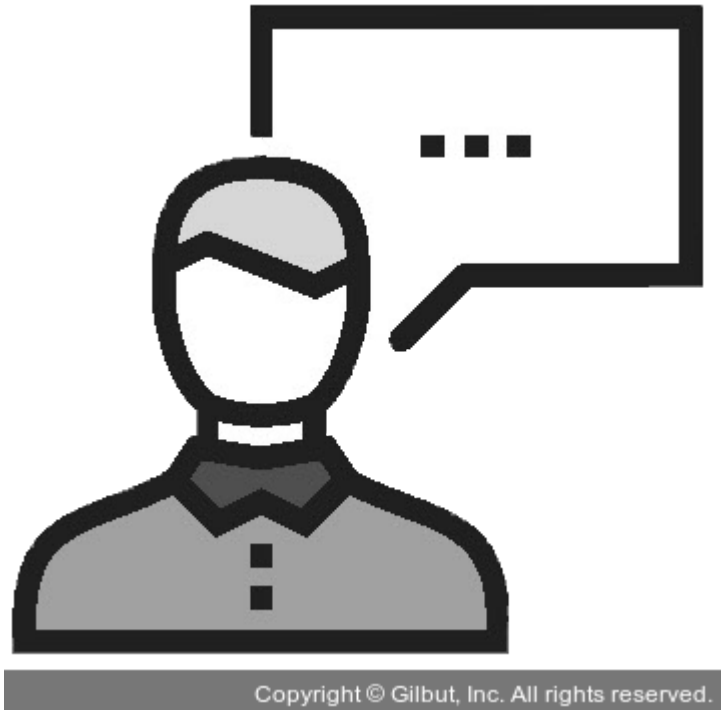
UNIT 15

테이블 형태의 데이터를 쉽게 다루도록 도와주는 pandas 라이브러리



⦿ 실습 데이터 Unit15/unit15-01.ipynb, age.csv

이번 장에서는 `numpy` 기반의 `pandas`라는 데이터 분석용 라이브러리에 대해 알아보겠습니다. `pandas` 라이브러리는 파이썬 데이터 분석에 가장 많이 쓰이는 라이브러리인데요. `pandas`라는 말을 처음 듣는 사람도 있겠지만, 이 책을 다 읽고 난 후 더 전문적인 데이터 분석을 하기 원한다면 반드시 만나게 될 겁니다.

`pandas` 라이브러리만을 다루는 전문 서적이 있을 정도로 내용이 방대하기 때문에, 이 책에서는 '`pandas` 라이브러리가 이런 거구나'를 느낄 정도로만 살펴보겠습니다. 그런 다음 Unit 14에서 만들어본 데이터 프로젝트를 `pandas` 라이브러리를 활용해 해결해 보며 그 유용함을 느껴보겠습니다.

먼저 `pandas`라는 라이브러리를 어떻게 사용할 수 있는지 아주 간단한 예를 소개하겠습니다.

1 위키피디아 데이터 엑셀로 저장하기

4년마다 열리는 올림픽은 국가별로 여러 종목의 선수들을 한번에 만날 수 있는 국제적인 행사입니다. `pandas` 라이브러리 실습을 위해 위키피디아 웹 페이지의 올림픽 메달 기록 데이터를 엑셀로 저장하는 방법을 알아보겠습니다.

먼저 위키피디아의 올림픽 메달 기록을 정리해둔 페이지로 이동해 볼까요? 구글 검색 창에 olympic medal 또는 olympic medal table이라고 입력합니다.



그림 15-1 구글에서 olympic medal로 검색한 결과

2 pandas란

`pandas`는 panel datas(패널 자료)의 약자로 파이썬을 활용한 데이터 분석에서 가장 많이 활용되는 라이브러리입니다.

Unit 13에 배웠던 `numpy`를 기반으로 만들어졌으며 데이터 분석을 위한 효율적인 데이터 구조를 제공하는데요. 그중 1차원 배열 형태의 데이터 구조를 Series라고 부르고, 2차원 배열 형태의 데이터 구조를 DataFrame이라고 부릅니다.


Copyright © Gilbut, Inc. All rights reserved.

표 15-1 1차원 배열 형태: Series

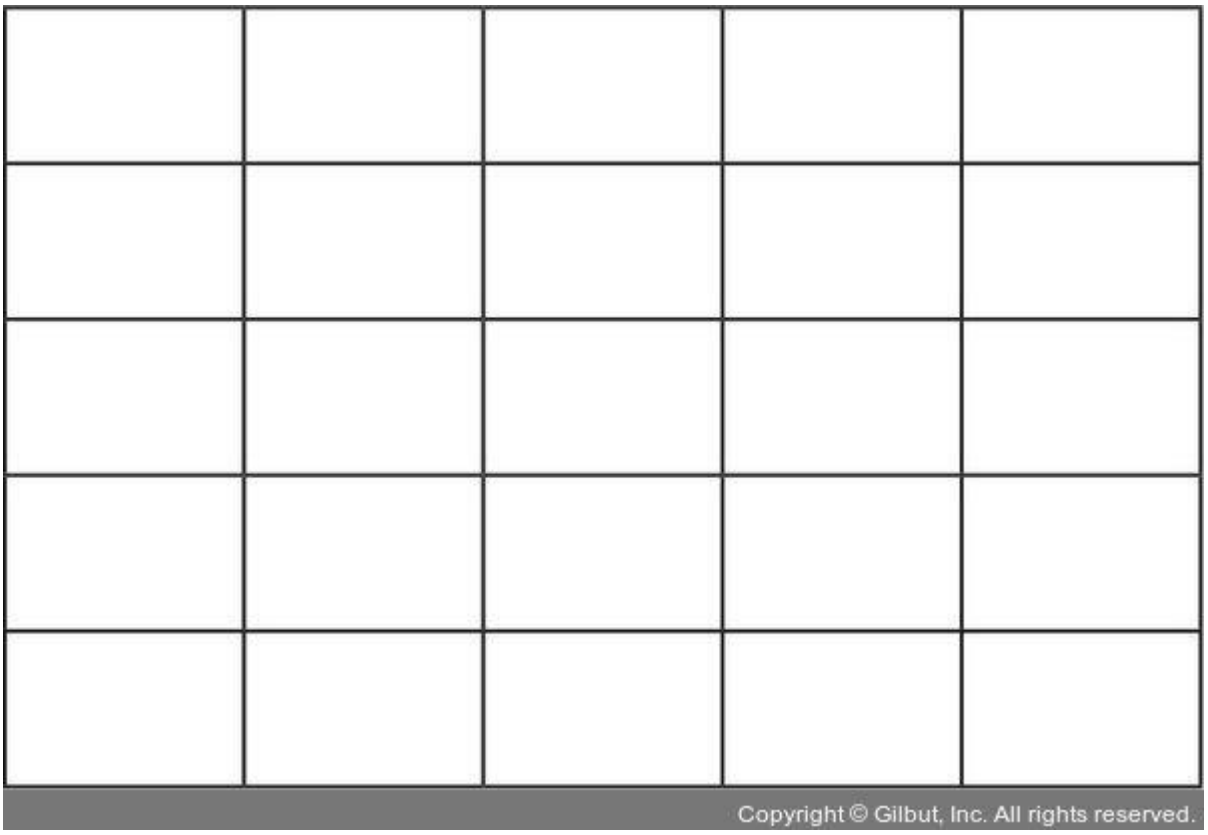

Copyright © Gilbut, Inc. All rights reserved.

표 15-2 2차원 배열 형태: DataFrame

이 책에서는 2차원 배열 형태인 DataFrame을 중점적으로 살펴보겠습니다. 사실 조금 전에 올림픽 표를 저장했던 `df`라는 변수도 DataFrame이라는 뜻으로 사용된 것이었습니다.

이 책에서는 주로 기본 파이썬 문법을 활용해서 많은 문제들을 해결했는데요. 이를 통해 데이터 구조나 데이터를 처리하는 방법에 대해서는 어느 정도 이해했을 것입니다. 이제 여기에 `numpy`나 `pandas` 같은 라이브러리를 활용하면 보다 효율적이고 전

문적인 데이터 분석을 할 수 있습니다. 이미 만들어진 함수를 활용하기 때문에 코드도 간결해지고요.

3 데이터 프레임 기초

`pandas`의 가장 기본적인 구조인 DataFrame에 대해 간략히 알아보겠습니다. 2차원 배열 형태의 데이터 프레임은 행과 열이 있고, 행을 구분해주는 인덱스(index)와 열을 구분해주는 컬럼(column)이 있습니다. 그래서 별도로 지정해주지 않으면 인덱스는 리스트처럼 정수로 설정이 되고 한 번 설정된 인덱스는 변경되지 않습니다.

다음과 같이 날짜 형태로 된 8개의 인덱스를 간단히 만들겠습니다.

```
import pandas as pd
index = pd.date_range('1/1/2000', periods=8)
print(index)
```

실행 결과

```
DatetimeIndex(['2000-01-01', '2000-01-02', '2000-01-03', '2000-01-04',
               '2000-01-05', '2000-01-06', '2000-01-07', '2000-01-08'],
               dtype='datetime64[ns]', freq='D')
```

4 pandas로 인구 구조 분석하기

지금까지 `pandas` 라이브러리 사용법을 살펴보았습니다. 이제 Unit 14에서 파이썬 기본 문법과 `numpy` 라이브러리를 활용해서 만들었던 데이터 프로젝트를 `pandas` 라이브러리로 구현해 보겠습니다. 알고리즘은 Unit 14에서 작성했던 것을 그대로 가져왔습니다.

1 | 데이터를 읽어온다.

❶ 전체 데이터를 총 인구수로 나누어 비율로 변환한다.

❷ 총인구수와 연령구간인구수를 삭제한다.

2 | 궁금한 지역의 이름을 입력받는다.

3 | 궁금한 지역의 인구 구조를 저장한다.

4 | 궁금한 지역의 인구 구조와 가장 비슷한 인구 구조를 가진 지역을 찾는다.

❶ 전국의 모든 지역 중 한 곳(B)을 선택한다.

❷ 궁금한 지역 A의 0세 인구 비율에서 B의 0세 인구 비율을 뺀다.

❸ ❷를 100세 이상 인구수에 해당하는 값까지 반복한 후 차이의 제곱을 모두 더한다.

❹ 전국의 모든 지역에 대해 반복하며 그 차이가 가장 작은 지역을 찾는다.

5 | 가장 비슷한 곳의 인구 구조와 궁금한 지역의 인구 구조를 시각화한다.

■ 1 | 데이터 읽어오기

먼저 데이터를 읽어옵니다. 지금까지는 `csv` 라이브러리를 사용했지만 `pandas`에서는 `read_csv()` 함수를 활용하면 됩니다. 지금까지의 방법과 다른 점은 인코딩 방식을 작성한다는 것인데요. 윈도에서 만든 csv 파일일 경우 `encoding`에 `cp949`라고 작성합니다. Unit 14에서 사용했던 age.csv 파일이 있는 폴더에 다음 코드를 저장하여 실행합니다.

```
import pandas as pd
df = pd.read_csv('age.csv', encoding='cp949', index_col=0)
df.head()
```

행정구역	총인구수	연령구간인구수	0세	1세	2세	3세	4세	5세	6세	7세	...	91세	92세	93세	94세	95세	96세	97세	98세	99세	100세 이상
서울특별시 (1100000000)	9857426	9857426	61253	70532	74322	72482	71688	78941	75386	75929	...	5892	4695	3589	3501	2569	1851	1436	1010	736	5519
서울특별시 종로구 (1111000000)	154770	154770	652	794	911	895	859	1046	909	1012	...	149	115	95	79	81	60	46	37	26	226
서울특별시 종로구 청운효자동 (1111051500)	13272	13272	81	82	90	105	97	116	117	116	...	12	14	7	7	8	3	5	1	1	17
서울특별시 종로구 사직동 (1111053000)	9441	9441	43	63	75	68	76	81	74	83	...	11	13	7	5	7	3	6	4	4	17
서울특별시 종로구 삼청동 (1111054000)	2907	2907	12	4	22	13	12	13	20	11	...	5	2	3	3	2	2	0	2	1	7

5 rows × 103 columns

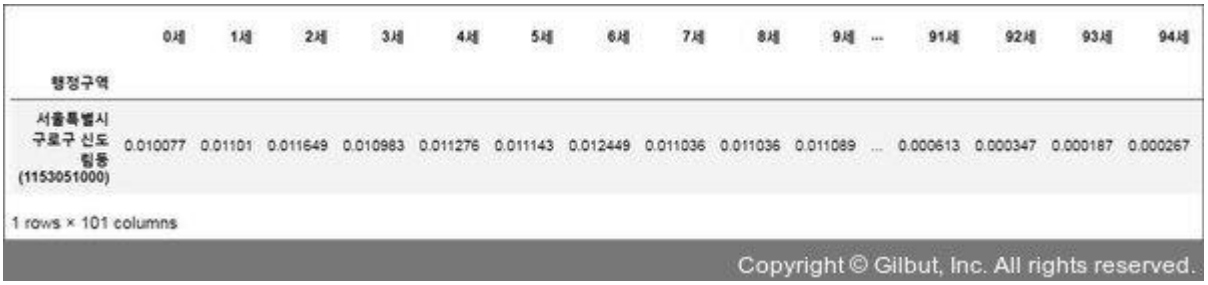
Copyright © Gilbut, Inc. All rights reserved.

그림 15-17 age.csv를 읽어온 결과

■ 2~3 | 궁금한 지역 이름 입력받고 해당 지역의 인구 구조 저장하기

`input()` 함수로 알고자 하는 지역의 이름을 입력받고 해당 지역의 인구 구조를 저장합니다. `df.index.str.contains()` 함수는 데이터 프레임의 인덱스 문자열에 원하는 문자열이 포함된 행을 찾아냅니다.

```
name = input('원하는 지역의 이름을 입력해주세요 : ')  # 2 | 지역 이름 입력
a = df.index.str.contains(name)  # 3 | 해당 행을 찾아서 해당 지역의 인구 구조를 저장
df2 = df[a]
df2
```



행정구역	0세	1세	2세	3세	4세	5세	6세	7세	8세	9세	...	91세	92세	93세	94세
서울특별시 구로구 신도림동 (1153051000)	0.010077	0.01101	0.011649	0.010983	0.011276	0.011143	0.012449	0.011036	0.011036	0.011089	...	0.000613	0.000347	0.000187	0.000267

1 rows × 101 columns

Copyright © Gilbut, Inc. All rights reserved.

그림 15-18 신도림동의 인구 구조를 출력한 결과

■ 4~5 | 궁금한 지역의 인구 구조와 가장 비슷한 인구 구조를 가진 지역 시각화하기

이제 데이터를 처리하는 가장 중요한 부분입니다. 여기에서는 각 알고리즘 코드를 개별로 살펴본 후, 마지막에 전체 코드를 실행하여 결과를 확인하겠습니다.

먼저 궁금한 지역의 연령별 비율과 다른 지역의 연령별 비율의 차이를 계산합니다. 그리고 궁금한 지역의 연령별 비율과 다른 지역의 연령별 비율의 차이를 계산합니다.

```
import numpy as np
# 4 | ❶ 궁금한 지역 A의 인구 비율에서 B의 인구 비율을 뺀다.
x = df.sub(df2.iloc[0], axis=1)
# 4 | ❷ A의 인구 비율에서 B의 인구 비율을 뺀 값의 제곱 값을 모두 더한다.
y = np.power(x, 2)
z = y.sum(axis=1)
```

Unit 14에서는 궁금한 지역의 연령별 비율과 다른 지역의 연령별 비율의 차이를 계산하여 가장 차이가 작은 지역 한 곳을 찾았습니다. 예를 들어, '신도림동'을 입력하면 '서울특별시 구로구 구로제1동' 한 곳의 결과만 볼 수 있었습니다. 하지만 `pandas`의 `sort_values()` 정렬 함수와 슬라이싱을 이용하면 상위 몇 개 지역까지 쉽게 찾을 수 있습니다.

```
i = z.sort_values().index[:5]  # 4 | ❸ 그 차이가 가장 작은 지역 5곳을 찾는다.
```